

Smart Airport Management System (SAMS)

Project Proposal for UCS503P

Submitted to: Dr. Anushka

Thapar Institute of Engineering and Technology

TEAM MEMBERS:

Ridhi Mahajan [1024240048]

Priyanka [1024240053]

Diya Saini [1024240057]

Ishika Jain [1024240066]

1 Higher-order goal

The proposed Smart Airport Management System (SAMS) enhances airport operational efficiency by integrating functions typically managed through separate and independently maintained systems. The objective is to support smoother, safer, and more predictable operations for passengers and airport personnel. The project converts coordination requirements across multiple airport functions into a centralized, measurable, and practically deployable software solution.

2 Time-to-value

The project prioritizes delivery of functional increments by initially developing the core flight-scheduling and dashboard workflow, validating its functionality, and subsequently extending the platform to additional modules.

Development follows an iterative approach with regular weekly milestones. The backend API, frontend dashboard, and operations modules are developed in parallel, with API contracts defined before implementation to enable independent work streams.

Initial success is assessed through functionality delivered in the first iteration, particularly the flight-management workflow and dashboard foundation. Subsequent iterations incorporate feedback obtained through simulated administrator, staff, airline-staff, passenger, visitor, and transport-operator roles.

3 Problem Statement

Modern airport operations involve several interconnected functions, including flight management, baggage handling, transportation, staff coordination, and visitor administration. When these activities are managed through independent systems, the absence of a unified operational view creates delays, duplicated effort, and difficulties in

maintaining consistent information. The major challenges addressed by the proposed system are:

Fragmentation: Flight, baggage, staff, visitor, and transportation information maintained in separate systems results in limited visibility, repeated data handling, and loss of operational context.

Delayed baggage tracking: Identifying and tracing lost or mishandled baggage becomes more difficult when baggage movements are not maintained through a unified tracking record.

Inefficient staff allocation: Shift and duty information may not be readily visible across operational departments, making it difficult to align staffing levels with peak operational periods.

Poor visitor coordination: Manual visitor registration and pass-issuance processes increase administrative effort and make systematic auditing more difficult.

Unorganized transport scheduling: Cab and shuttle operations may not be sufficiently synchronized with flight arrival and departure schedules, resulting in avoidable coordination gaps.

Why this matters now: As passenger volumes and airport activities increase, even relatively small coordination delays among flight operations, baggage handling, staffing, and ground transportation accumulate into noticeable service disruptions. A centralized management platform reduces this coordination overhead by providing a common operational view and consistent workflow across the relevant functions.

3.1 Overview

The proposed solution is a web-based Smart Airport Management System that brings key airport operational activities into a centralized platform through the following modules:

Flight Management: Scheduling, gate allocation, and flight-delay/status updates.

Luggage Management: Baggage registration, tracking, and lost-luggage reporting.

Visitor Management: Online and on-site visitor registration with digital pass generation.

Transportation Management: Scheduling and coordination of cab and shuttle services with flight timings.

Staff Management: Shift allocation, attendance records, and duty tracking.

Airport Dashboard: Consolidated display of live operational statistics, exceptions, and reports across the system.

3.2Core workflow

Airline or airport staff create or update flight information, including schedule, gate, and operational status. Updated information is reflected in the central dashboard.

Baggage is registered against the corresponding flight and passenger and assigned a trackable identification value. Movement is recorded through defined status stages: checked-in, in-transit, arrived, claimed, or lost, with associated timestamps.

Visitors complete registration online or at the airport and receive a role-appropriate digital pass valid for a predefined time period.

Transportation operators access relevant flight arrival and departure information and schedule shuttle or cab services accordingly.

Staff managers allocate shifts and duties. Attendance information is recorded and incorporated into the operational dashboard.

Airport administrators access a consolidated dashboard presenting flight information, baggage exceptions, staff availability, visitor counts, and transportation demand.

3.3 Operational constraints for an educational, single-semester setting

The application uses responsive web principles and remains usable on low-to-mid-range computing devices.

The system does not depend on physical hardware, CCTV infrastructure, or biometric systems. Where required, scanning or sensor-related events are simulated.

The solution is deployable using commonly available web-hosting services and a managed MongoDB environment.

Ticket booking and payment-gateway functionality are outside scope. The system is an airport operational management platform.

3.4 Solution Approach

The implementation focuses on administrative workflows and system integration. Development is organized around three parallel tracks:

Backend API: Database schema and REST endpoints are implemented first, with API contracts documented to support frontend and module development.

Frontend + Dashboard: The dashboard and role-specific interfaces are built against the defined API contracts.

Operations Modules: Flight, luggage, visitor, staff, and transportation management are implemented using the backend API.

3.5 Scheduling and tracking logic

Flight and baggage operations use deterministic, rule-based logic:

Gate-allocation conflicts are identified using time-window overlap checks against existing flight assignments.

Baggage movement is represented through a state machine with defined transitions: checked-in → in-transit → arrived → claimed/lost.

Conflicting or duplicate flight and staff-shift entries are flagged for review. Safety-relevant conflicts are not automatically resolved.

3.6 Web architecture

The system follows a 3-tier web architecture:

Frontend: React.js with Vite, Tailwind CSS, and React Router provides role-specific interfaces for administrators, staff, airline personnel, passengers, visitors, and transportation operators. Axios handles API communication, and Recharts renders dashboard visualizations.

Backend API: Node.js with Express.js provides RESTful endpoints for individual modules and implements JWT-based authentication with Role-Based Access Control (RBAC).

Database: MongoDB with Mongoose maintains collections for flight information, baggage records, visitor passes, staff shifts, transportation bookings, and user accounts.

4 Evaluation Criterion

We define success using measurable metrics tied to the core workflow.

4.1 Primary evaluation metric

Time-to-Status-Reflection (TSR): time between an operational event occurring (flight status change, baggage scan, shift update) and its reflection on the relevant dashboard/view.

- **Target:** median TSR < 5 seconds during pilot testing.
- **Attribution:** measured from database event timestamps versus dashboard/API response timestamps.

4.2 Secondary evaluation metrics

- **Gate/shift conflict detection rate:** percentage of overlapping flight or shift bookings correctly flagged before confirmation.
- **Lost-luggage traceability:** percentage of baggage-status queries resolved from logged state without manual lookup.
- **Visitor pass issuance time:** average time from visitor registration to pass generation.
- **Role-based access correctness:** percentage of endpoints that correctly enforce RBAC under test (no unauthorized access).
- **Reliability:** availability of staging/production for login and core operations during test windows (e.g., 99% uptime in pilot).

4.3 Pilot validation plan

- Recruit 10-15 simulated users across roles (administrator, staff, airline staff, passenger, visitor, transport operator), since access to a live airport is infeasible for a course project.
- Run a scripted set of scenarios (flight delay, baggage mishandling, visitor check-in, shift change, transport booking) and log metrics before/after workflow completion.

5 Scalability

The proposition should scale in both theoretical and practical senses.

1. **Scalable (theoretically):** The system should support growth in flight volume, baggage records, and concurrent dashboard usage.

- decoupling components (React frontend / Express backend / MongoDB),
 - indexing MongoDB collections on high-query fields (flight ID, baggage tag, shift date),
 - using stateless, JWT-authenticated API design so backend instances can be scaled horizontally.
2. Operationally deployable (practically): The system should run on common web hosting.
- managed MongoDB database (e.g., MongoDB Atlas),
 - containerized or platform-hosted deployment for frontend and backend services,

6 Engine availability heuristic

A set of “engines” (tooling/services) is available to implement this as a web-based project:

- React.js for the frontend, with role-specific dashboards and forms.
- Node.js and Express.js for REST API development.
- MongoDB for a flexible, document-oriented data store.
- JWT and Role-Based Access Control for authentication and authorization.
- Test framework (e.g., Jest) for unit/integration tests.

We will use these components to focus on workflow design, integration correctness, and measurement rather than inventing new infrastructure.

7 Project deliverables

- Authentication with JWT + RBAC for all six user roles
- Flight Management: scheduling, gate allocation, delay updates
- Luggage Management: baggage tracking and lost-luggage reporting
- Visitor Management: registration and digital visitor passes
- Staff Management: shifts, attendance, duty tracking
- Transportation Management: cab/shuttle scheduling
- Airport Dashboard: skeleton view showing live flight statistics
- Basic logging and timestamps for TSR measurement
- Expanded dashboard: cross-module reports and analytics
- Improved search/filtering and indexed queries for scale readiness

8 Risks and mitigations

- **Module integration:** six modules touching shared entities (flights, users) risk inconsistent state; mitigate with a clearly-defined shared data schema and API contracts agreed upfront.
- **Database design:** poor schema choices in MongoDB can hurt query performance at scale; mitigate by modeling core entities and access patterns before implementation, and indexing early.
- **Real-time updates:** dashboard staleness can undermine trust; mitigate with polling or lightweight WebSocket updates on high-priority events (flight delays, gate changes).
- **Role-based security:** misconfigured RBAC can expose sensitive operations; mitigate with centralized middleware for role checks and automated authorization tests.

9 Summary

This project proposes a deployable web platform, the **Smart Airport Management System**, to centralize flight, luggage, visitor, staff, and transport operations that are otherwise fragmented across siloed systems.